# Programming Assignment (Due date: Nov 11, 2026)

Start Assignment

- Due Wednesday by 11:59pm
- Points 6
- Submitting a file upload
- File Types py, c, cc, cpp, java, txt, pdf, and md
- Available Oct 19 at 9am - Dec 18 at 11:30pm

**CS3201 2026 Programming Assignment**

# Online Image Description System

> ### Download the Assignment Files
>
> **image-description-assignment.zip**
> **(https://canvas.cityu.edu.hk/courses/70555/files/16277107/download?download_frd=1)**
>
> Includes the server template, setup scripts, model files and test images.

**This is a group assignment. Each group may have up to three students.** See Section 6 for group registration and submission details.

## 1. Assignment Description and Your Tasks

In this project, you are required to develop a **client-server application** - the **Online Image Description System** that consists of a server and multiple clients, where each client can send its requests to the server through the IPv4 TCP connection, and the server can listen client requests on **TCP port 18333 a**nd makes response to the requests. All client requests are related to images, for exmaple, a client uploads an image and sends the image to the server. There is an AI model deployed in the server side which can accept an image for processing and generate an output - the English description about the image, e.g., classifying the image into one of different categories, e.g., a cat or a dog, etc.

There are six commands in client side to be implemented which are `UPLOAD`, `LIST`, `DESCRIBE`, `DOWNLOAD`, `REMOVE` and `QUIT`. Specifically, the client uses the `UPLOAD` command to send its image to the server, uses the `LIST` command to view the uploaded image IDs and their description status, use the `DESCRIBE` command to request an English description of a selected image from the server, uses the `DOWNLOAD` command to retrieve its uploaded image file, uses the `REMOVE` command to delete an image from the server, and the `QUIT` command to close the client current connection with the server. In Section 3 of this project description, more detailed descriptions

of these commands  are given through an example of a client interaction with the server, using the six commands.

**Your job is to complete (i) the five unfinished functions in the server side; and (ii) the entire client socket  (i.e., the six commands) in Python, C/C++ or Java.**

Notice that Image management and local CPU inference are supplied. All code comments, prompts, error messages and all submitted materials must be in English.

## Task 1: Implement  the five unfinished functions in the server side

Complete five functions marked `# TODO` in Part 1 of `ImageDescriptionServer.py`. Do not change names and arguments of the following function. Follow their docstrings and the message rules in Section 4 of this project description.

Five Socket functions

| Function | Main task |
|---|---|
| `create_listener` | Create a TCP listening Socket. |
| `accept_client` | Accept a client connection. |
| `receive_message` | Receive one complete message. |
| `send_message` | Send one complete message. |
| `close_client` | Close the client connection. |

You may use the provided `recv_exact` function when implementing `receive_message`. Parts 2, 3 and 4 of `ImageDescriptionServer.py` handle client connections, process commands and run the local AI model. These parts are already implemented, and you do not need to modify them. If you need additional Python standard-library modules, add the corresponding `import` statements.

## Task 2: Develop the client socket in  the client side

Your job is to implement six commands `UPLOAD`, `LIST`, `DESCRIBE`, `DOWNLOAD`, `REMOVE` and `QUIT` over **one persistent IPv4 TCP connection in client side**. Follow the command requirements in Section 3 of this project and the message format and exchange order in Section 4 of this project description. Use standard Socket APIs and access the server through TCP. Python clients must use only the standard library.

**Follow the instructions in Section 2 of the project description to set up the Python environment and run the server program.**

# 2. Using the Provided Server Program

Extract `image-description-assignment.zip` (https://canvas.cityu.edu.hk/courses/70555/files/16277107/download?download_frd=1) to a local folder. It contains `ImageDescriptionServer.py`, two setup scripts, dependency lists, `test_images` and `models/smolvlm-256m`. Keep these files together and leave the folder in that location.

## 2.1 First Time Setup

The provided setup script uses uv to install Python, create a virtual environment and install the required CPU dependencies with fixed versions. This provides a consistent runtime environment for all students. **Setup requires internet access.**

Open Terminal (macOS/Linux) or PowerShell (Windows) and change to the extracted folder. **Adjust these example paths for your computer.**

**macOS/Linux Terminal**

```
cd ~/Downloads/image-description-assignment
bash setup_env.sh
```

**Windows PowerShell**

```
cd "C:\Users\YourName\Downloads\image-description-assignment"
.\setup_env.bat
```

Wait for `Setup complete`. The setup scripts are provided; do not modify them. Setup stores its tools, Python and dependencies in `.tools`, `.runtime` and `.venv`.

## 2.2 Check the AI Component Before Coding

This check verifies that the dependencies installed in Section 2.1 can run the AI component and produce an image description. Use `test_images/cat.jpg`, one of the three provided sample images, or your own JPEG or PNG image.

**macOS/Linux**

```
.venv/bin/python ImageDescriptionServer.py --check-ai ./test_images/cat.jpg
```

**Windows**

```
.\.venv\Scripts\python.exe ImageDescriptionServer.py --check-ai .\test_images\cat.jpg
```

A successful check prints an image description.

## 2.3 Start the Server Program

**Complete all five TODO functions before performing network testing.** AI inference may take a while. Use the mock mode to debug Socket communication quickly; it returns a fixed description without running the AI model.

### Start in mock mode — macOS/Linux

```
.venv/bin/python ImageDescriptionServer.py --backend mock
```

### Start in mock mode — Windows

```
.\.venv\Scripts\python.exe ImageDescriptionServer.py --backend mock
```

The following output means the server is ready to accept connections:

```
READY host=0.0.0.0 port=18333
```

- On the same computer, start your client in another terminal and connect to `127.0.0.1:18333`.
- From another computer, connect to the server computer's IPv4 address at port `18333`. The default listening address `0.0.0.0` includes all IPv4 interfaces.
- Use the `.venv` interpreter for a Python client.

### Run the Server with the AI Model

Stop the mock server with `Ctrl+C`, then run the command for your system.

### macOS/Linux

```
.venv/bin/python ImageDescriptionServer.py
```

### Windows

```
.\.venv\Scripts\python.exe ImageDescriptionServer.py
```

When `READY` appears, reconnect your client and test all six commands in Section 3.

## 2.4 Troubleshooting

- **Setup fails:** check your internet connection and rerun the setup script.
- **Missing interpreter or module:** rerun setup. If it reports an incompatible uv or `.venv`, contact the TA.
- **Missing or damaged model files:** contact the TA.

- **TODO message:** finish the named function.
- **Port in use:** stop the previous server.

For unresolved problems, email the TA using the details in Section 6.

## 3. Client Commands and Required Behavior

The TCP port on the server side is `18333` by default. Implement the following six commands.

Client commands

| Command | Subsequent terminal input | Required result |
|---|---|---|
| `UPLOAD` | Local JPEG/PNG path | Upload the file and display its assigned image ID. |
| `LIST` | None | Display IDs, filenames, upload times (HH:MM:SS) and description-cache status. |
| `DESCRIBE` | Image ID | Display the image description. |
| `DOWNLOAD` | Image ID, then local output path | Save the original bytes to a new local file. |
| `REMOVE` | Image ID | Remove that image and its cached description. |
| `QUIT` | None | Display OK, close the connection and exit. |

### 3.1 Reference Client Session

The example below shows a sample client session. **Your client must follow its input and output format.** Enter each command and its required values on separate lines. Marks may be deducted for not following this input format or the `client:` and `server:` prefixes.

- `./missing.jpg` does not exist. The client displays the error without uploading it.
- `./invalid.jpg` exists but is not a valid JPEG or PNG image. The server returns an error after receiving it.

```
$ .venv/bin/python ImageDescriptionClient.py 127.0.0.1 18333
client: UNKNOWN
server: ERROR - Command not understood
client: UPLOAD
client: ./missing.jpg
client: ERROR - File not found
client: UPLOAD
```

```
client: ./invalid.jpg
server: ERROR - Invalid image
client: UPLOAD
client: ./test_images/cat.jpg
server: OK 0000
client: LIST
server: OK
server: IMAGE ID: 0000 | FILE: cat.jpg | TIME: 00:07:39 | DESCRIPTION: no
client: DESCRIBE
client: 9999
server: ERROR - Invalid ID
client: DESCRIBE
client: 0000
server: OK
server: In this image we can see a cat lying on the floor.
client: LIST
server: OK
server: IMAGE ID: 0000 | FILE: cat.jpg | TIME: 00:07:39 | DESCRIPTION: yes
client: DOWNLOAD
client: 9999
client: missing_download.jpg
server: ERROR - Invalid ID
client: DOWNLOAD
client: 0000
client: cat_download.jpg
server: OK cat.jpg
client: Saved to cat_download.jpg
client: REMOVE
client: 9999
server: ERROR - Invalid ID
client: REMOVE
client: 0000
server: OK
client: LIST
server: OK
server: No images
client: QUIT
server: OK
```

### 3.2 File and Session Rules

- Upload JPEG or PNG images up to **5 MiB and 20 million pixels**. Server storage is limited to 20 images and 50 MiB.
- Upload and download paths refer to files on the **client computer**. Send only the upload filename (at most 255 characters) and image data; do not send local paths.
- `DOWNLOAD` must not overwrite an existing file. Display a save confirmation only after receiving and saving the complete file.

- IDs start at `0000` and are not reused. All clients share one list. Restarting the server clears images, cached descriptions and IDs.
- The server generates a description on the first `DESCRIBE` request for an image and reuses it for later requests.
- After an unknown command, invalid image ID or invalid image, display the error reply and continue using the same connection. `QUIT` closes only that client's connection.

Before uploading, check that the file exists, can be read and is no larger than 5 MiB. If a path is invalid, display an error message. **Print the server's actual reply in the terminal**, rather than fixed values copied from Section 3.1.

### 3.3 Try the Demo

Demo programs help you explore the six commands before implementing your own programs. They run without installing Python or third-party libraries.

Download the Demo for your system and extract its ZIP to a local folder:

- **Windows x64:** [windows-x64.zip (https://canvas.cityu.edu.hk/courses/70555/files/16277151/download?download_frd=1)](https://canvas.cityu.edu.hk/courses/70555/files/16277151/download?download_frd=1)
- **macOS Apple Silicon:** [macos-arm64.zip (https://canvas.cityu.edu.hk/courses/70555/files/16277148/download?download_frd=1)](https://canvas.cityu.edu.hk/courses/70555/files/16277148/download?download_frd=1)
- **Ubuntu x64:** [ubuntu-x64.zip (https://canvas.cityu.edu.hk/courses/70555/files/16277149/download?download_frd=1)](https://canvas.cityu.edu.hk/courses/70555/files/16277149/download?download_frd=1)

Open two terminals in the extracted folder. Start the server program in the first terminal. When `READY` appears, start the client in the second terminal.

**Windows PowerShell — server terminal**

```
.\ImageDescriptionServerDemo.exe
```

**Windows PowerShell — client terminal**

```
.\ImageDescriptionClientDemo.exe 127.0.0.1 18333
```

**macOS/Ubuntu — server terminal**

```
./ImageDescriptionServerDemo
```

**macOS/Ubuntu — client terminal**

```
./ImageDescriptionClientDemo 127.0.0.1 18333
```

The Demo server uses port `18333`. To simplify setup and help you debug Socket communication quickly, it does not run a real AI model. Every successful `DESCRIBE` returns:

```
server: OK
server: This is a demo server. This is a test description.
```

Try the six commands using Section 3.1's input and output format, with your own JPEG/PNG image. Use `QUIT` to close the client and `Ctrl+C` to stop the Demo server.

### 3.4 Suggested Debugging Order

Work in small steps and check each step before adding the next feature:

1. Use mock mode. Complete the five server functions, then check that `LIST` and `QUIT` work.
2. Add `UPLOAD` and use `LIST` to check the assigned image ID.
3. Add `DESCRIBE` and `REMOVE`. Check the description status before and after a description is generated.
4. Add `DOWNLOAD`. Check that the saved file has the same bytes as the uploaded file, then try the error examples in Section 3.1.
5. Switch to real AI and test the completed client with your server.

## 4. Message Format and Exchange Order

TCP transfers data as a stream of bytes, so the receiver must determine how much data belongs to each message. **For this project, we define a message format in which your program adds a length header before the message data.** The header tells the receiver how many data bytes to read for one complete message.

### 4.1 Length Header and Data

Each message contains a **10-byte length header followed by the playload data**. Write the data length as ten ASCII decimal digits, adding zeros at the beginning when needed.

- Count data bytes only, excluding the header. Valid lengths are 1 to 5,242,880 bytes.
- Encode text as UTF-8 and count its encoded bytes.
- Send images as their original file bytes.

For example, `HELLO` contains five bytes:

```
0000000005HELLO
```

Collect the 10 header bytes, read the length as 5, then collect five data bytes, the message then completes.

## 4.2 Terminal Input and Socket Messages

Section 3.1 shows what the user types. The client converts that input into Socket messages using Section 4.1's format. For example:

```
client: UPLOAD
client: ./test_images/cat.jpg
```

The client reads `./test_images/cat.jpg` on the client computer. It sends the filename `cat.jpg` with the command. The text `UPLOAD cat.jpg` contains 14 bytes, so the first message is:

```
0000000014UPLOAD cat.jpg
```

Next, send the original image in bytes in the second message, with a separate length header. After sending both messages, receive and display the server's reply.

## 4.3 Command Exchange

Each message below has its own length header. Command text must not exceed 1,024 bytes. Replace `<filename>` and `<image_id>` with actual values. **Receive all replies to one command before sending the next command.**

Command exchange

| Command | What the client sends | What the client receives and does |
|---|---|---|
| `UPLOAD` | 1. Text: `UPLOAD <filename>`<br>2. Original image bytes.<br>Send both messages before waiting for a reply. | Receive and display one text reply: `OK <image_id>` or `ERROR - <reason>`. |
| `LIST` | One text message: `LIST` | Receive one text reply: `OK` followed by the list or `No images`. Display all lines. |
| `DESCRIBE` | One text message: `DESCRIBE <image_id>` | Receive one text reply: `OK` followed by the description, or an error. Display all lines. |
| `DOWNLOAD` | One text message: `DOWNLOAD <image_id>` | Receive a text reply. After `OK <filename>`, receive an image message and save its bytes. After an error, display it; no image message follows. |

| Command | What the client sends | What the client receives and does |
| --- | --- | --- |
| `REMOVE` | One text message: `REMOVE <image_id>` | Receive and display `OK` or an error. |
| `QUIT` | One text message: `QUIT` | Receive and display `OK`. Close the Socket and exit. |

`LIST` and `DESCRIBE` may return several text lines in **one message**, with a newline after `OK`. Receive the complete message, then display each line with the `server:` prefix.

## 5. Marking Scheme

TAs will assess your programs as submitted and do not help you to fix bugs. **Your report must clearly explain how to compile (if needed) and run your code.** The marking scheme below applies only if the TAs can successfully run your programs using the instructions in your report.

Marking scheme

| Component | Main requirements | Marks |
| --- | --- | --- |
| Connection setup | Create, accept and close TCP connections. | 10 |
| Message exchange | Send and receive complete messages using length headers. | 10 |
| UPLOAD | Upload an image and display its assigned ID. | 10 |
| LIST | Display uploaded images and their description status. | 10 |
| DESCRIBE | Request and display an image description. | 10 |
| DOWNLOAD | Save a downloaded image with its original bytes. | 10 |
| REMOVE | Remove the selected image and display the reply. | 10 |
| QUIT | Close the client connection and exit. | 10 |
| Basic error handling | Display errors for invalid requests or local files; continue when possible. | 10 |
| PDF report | Submit a PDF report. | 10 |
| **Total** | | **100** |

## 6. Submission and Getting Help

**Number of students in each group:** Up to 3 students
**Submission deadline:** 11 November 2026, 11:59 PM (Hong Kong time)
Marks may be deducted for late submissions.

**Group registration:** Before submitting, open **People** in Canvas and click the **Groups** tab. One member of your team should first join an empty group from **Project Group 001** to **Project Group 200**. The other team members must then join the same group. **Do not create a new group.** All members of your team must join the same group. The assignment is graded as a group, and all members in one group receive the same mark.

Each group must submit one set of the files listed below directly on Canvas. **Upload the code files and your PDF project report separately.** Only one member in your group submits the code files and the pdf report on behalf of your group.

- The completed server code, client source files and required build files, including a Makefile if applicable.
- A brief PDF report in English with at least the information below.

1. **Member information:** names and student IDs of all group members. State the operating system and client language/tool version.
2. **How to run:** exact commands to build (if needed) and start the server and client. Include any setup needed for the client.
3. **Results:** screenshots from your own runs showing the commands you completed and their output.

**File names**

Name the server `ImageDescriptionServer_<student_IDs>.py`, the client `ImageDescriptionClient_<student_IDs>.<extension>` (`py`, `cpp` or `java`), and the report `Report_<student_IDs>.pdf`. Replace `<student_IDs>` with all members' IDs, separated by underscores.

Example for two students using Python:

```
ImageDescriptionServer_51234567_57654321.py
ImageDescriptionClient_51234567_57654321.py
Report_51234567_57654321.pdf
```

Submit only the source/build files and PDF report. **DO NOT submit** models, `.venv`, `.tools`, `.runtime`, `setup.log` or executable files. The assignment is graded as a group, and all members receive the same mark.

**Getting help**

For questions or problems, email TA **ZHANG Yizhun** at yzhang5624-c@my.cityu.edu.hk (mailto:yzhang5624-c@my.cityu.edu.hk) with a brief description and relevant error screenshots or information.